

```

import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;

class Result {

    /**
     * Complete the 'matrixRotation' function below.
     *
     * The function accepts following parameters:
     * 1. 2D_INTEGER_ARRAY matrix
     * 2. INTEGER r
     */

    public static void matrixRotation(List<List<Integer>> matrix, int r) {
        int m = matrix.size();
        int n = matrix.get(0).size();

        int layers = Math.min(m, n) / 2;

        for (int layer = 0; layer < layers; layer++) {
            // Extract elements of this layer into a list
            List<Integer> elements = new ArrayList<>();

            // top row
            for (int j = layer; j < n - layer; j++) {
                elements.add(matrix.get(layer).get(j));
            }
            // right column
            for (int i = layer + 1; i < m - layer - 1; i++) {
                elements.add(matrix.get(i).get(n - layer - 1));
            }
            // bottom row
            for (int j = n - layer - 1; j >= layer; j--) {
                elements.add(matrix.get(m - layer - 1).get(j));
            }
        }
    }
}

```

```

// left column
for (int i = m - layer - 2; i > layer; i--) {
    elements.add(matrix.get(i).get(layer));
}

int len = elements.size();
int rot = r % len;

// Rotate the list
List<Integer> rotated = new ArrayList<>();
for (int k = 0; k < len; k++) {
    rotated.add(elements.get((k + rot) % len));
}

// Put rotated elements back into matrix
int idx = 0;
// top row
for (int j = layer; j < n - layer; j++) {
    matrix.get(layer).set(j, rotated.get(idx++));
}
// right column
for (int i = layer + 1; i < m - layer - 1; i++) {
    matrix.get(i).set(n - layer - 1, rotated.get(idx++));
}
// bottom row
for (int j = n - layer - 1; j >= layer; j--) {
    matrix.get(m - layer - 1).set(j, rotated.get(idx++));
}
// left column
for (int i = m - layer - 2; i > layer; i--) {
    matrix.get(i).set(layer, rotated.get(idx++));
}
}

// Print the rotated matrix
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        System.out.print(matrix.get(i).get(j) + " ");
    }
    System.out.println();
}
}
}

```

```

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new
InputStreamReader(System.in));

        String[] firstMultipleInput = bufferedReader.readLine().replaceAll("\\s+$", "").split(" ");

        int m = Integer.parseInt(firstMultipleInput[0]);

        int n = Integer.parseInt(firstMultipleInput[1]);

        int r = Integer.parseInt(firstMultipleInput[2]);

        List<List<Integer>> matrix = new ArrayList<>();

        IntStream.range(0, m).forEach(i -> {
            try {
                matrix.add(
                    Stream.of(bufferedReader.readLine().replaceAll("\\s+$", "").split(" "))
                        .map(Integer::parseInt)
                        .collect(toList())
                );
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        });

        Result.matrixRotation(matrix, r);

        bufferedReader.close();
    }
}

```

Your Matrix Layer Rotation submission got 80.00 points. [Share](#) [Post](#)

You are now 75 points away from the 3rd star for your problem solving badge.

[Try the next challenge](#) | [Try a Random Challenge](#)

Problem Submissions Leaderboard Discussions Editorial

You made this submission a few seconds ago.

Score: 80.00 Status: **Accepted**

People who solved Matrix Layer Rotation attempted this next:

Almost Integer Rock Garden

Finish building a rock garden so that the sum of distances from each stone to the origin is an almost integer.

[Solve Challenge](#)

Submitted Code

Language: Java 8 [Open in editor](#)

```
1 import java.io.*;
2 import java.math.*;
3 import java.security.*;
4 import java.text.*;
5 import java.util.*;
6 import java.util.concurrent.*;
```

- NEED HELP?
- [View discussions](#)
 - [View editorial](#)
 - [View top submissions](#)